BY KITTU K PATEL

VERILOG INTERVIEW QUESTION

Verilog Interview Questions & Answers

Prepared by Kittu Patel

September 11, 2025

Contents

1 Verilog Interview Questions **5**

Chapter 1

Verilog Interview Questions

Introduction

This chapter contains 50 carefully curated Verilog interview questions with detailed solutions. Each question is presented with a clear answer, example code, and practical pro tips.

Q1. What is the difference between `wire` and `reg` in Verilog?

Answer: A `wire` is a net used to connect components and cannot store values. A `reg` is a variable that can hold a value and is usually used in procedural blocks.

Code

```
wire a;
reg b;

assign a = b & c;

always @(posedge clk)
  b <= d;
```

Pro Tip

In SystemVerilog, prefer using `logic` which can replace both `wire` and `reg`.

Q2. Explain blocking (=) vs non-blocking (<=) assignments. When to use each?

Answer: - Blocking assignments (=) execute sequentially. - Non-blocking assignments (<=) schedule updates for the end of the time step. Use blocking for combinational logic and non-blocking for sequential logic.

Code

```
always @(posedge clk) begin
  q1 <= d;    // non-blocking
  q2 <= q1;
end
```

Pro Tip

Never mix blocking and non-blocking in the same always block.

Q3. What does `always @(*)` mean and why is it used?

Answer: `always @(*)` automatically infers the sensitivity list for combinational logic. It helps prevent simulation mismatches by ensuring all inputs are considered.

Code

```
always @(*) begin
  case(sel)
    2'b00: y = a;
    2'b01: y = b;
    default: y = 0;
  endcase
end
```

Pro Tip

In SystemVerilog, use `always_comb` for stricter checks and better readability.

Q4. What is the difference between `initial` and `always` blocks in Verilog?

Answer: - `initial`: Executes only once at the start of simulation. Often used for testbench stimulus or initialization. - `always`: Executes repeatedly whenever triggered by its sensitivity list. Used for hardware description.

Code

```
initial begin
  clk = 0;
end

always #5 clk = ~clk;  // toggles forever
```

Pro Tip

Use `initial` only in testbenches; avoid it in synthesizable RTL.

//

//

Q5. Explain `case`

Answer: - `case`: Performs exact bit-by-bit matching. - `casex`: Treats both `x` and `z` as wildcards (don't care). - `casez`: Treats only `z` (high-impedance) as wildcard.

Code

```
casez(sel)
  2'b1?: out = a;   // ? means don't care
  2'b01: out = b;
endcase
```

Pro Tip

Prefer `casez` for synthesis safety. Avoid `casex` to prevent unintended matches.

Q6. What are `parameters` in Verilog and why are they used?

Answer: Parameters are constants that make modules reusable and configurable without rewriting code. They allow designers to define widths, delays, or functional settings inside modules.

Code

```
module counter #(parameter WIDTH=8)
   (input clk, output reg [WIDTH-1:0] count);

  always @(posedge clk)
    count <= count + 1;

endmodule
```

Pro Tip

Use parameters for design reusability. In SystemVerilog, `localparam` prevents override.

Q7. What is the difference between `fork-join` and `fork-join_any`?

Answer: - `fork-join`: All parallel processes must finish before execution continues. - `fork-join_any`: Execution continues as soon as any one process completes.

Code

```
fork
  #5 a = 1;
  #10 b = 1;
join    // waits 10 time units

fork
  #5 a = 1;
  #10 b = 1;
join_any // resumes after 5 time units
```

Pro Tip

Avoid using `fork-join` in synthesizable RTL; it is only for testbenches.

Q8. What is a sensitivity list and why use `always @(*)`?

Answer: A sensitivity list defines which signals trigger an `always` block. `always @(*)` automatically includes all right-hand side signals, preventing simulation mismatches.

Code

```
always @(*) begin
  y = a & b; // sensitive to a, b automatically
end
```

Pro Tip

In SystemVerilog, use `always_comb` for even stricter rules and better clarity.

Q9. Explain the difference between `posedge` and `negedge`.

Answer: - `posedge`: Triggered on rising edge of a signal ($0 \rightarrow 1$). - `negedge`: Triggered on falling edge of a signal ($1 \rightarrow 0$). Both are commonly used in clocked flip-flops.

Code

```
always @(posedge clk)
  q <= d; // rising edge

always @(negedge clk)
  q <= d; // falling edge
```

Pro Tip

Most flip-flops are designed with `posedge clk`; use `negedge` for special cases like DDR.

Q10. What are compiler directives in Verilog?

Answer: Compiler directives are special instructions (starting with `) that control simulation and compilation. Examples include: - `define` for macros - `include` for including files - `timescale` for setting time unit and precision

Code

```
`define WIDTH 8
`timescale 1ns/1ps

module top;
  reg [`WIDTH-1:0] data;
endmodule
```

Pro Tip

Use macros for global definitions, but prefer parameters for module-specific constants.

Q11. What is the purpose of the `initial` block in Verilog?

Answer: The `initial` block is executed only once at the start of the simulation. It is mainly used for: - Initializing signals - Generating stimulus for testbenches - Setting up conditions before simulation begins
Unlike `always` blocks, `initial` blocks cannot be synthesized for hardware.

Code

```
reg clk;
initial begin
  clk = 0;
  #5 clk = 1;
  #10 clk = 0;
end
```

Pro Tip

Use `initial` blocks only in testbenches, not in synthesizable RTL.

Q12. What is the difference between `$display` and `$monitor`?

Answer: - `$display`: Prints the values once at the time it is executed. - `$monitor`: Continuously tracks and prints the values whenever they change. Both are simulation-only system tasks and not synthesizable.

Code

```
initial begin
  $display("Time=%0t a=%b b=%b", $time, a, b);
  $monitor("Monitor at %0t: a=%b b=%b", $time, a, b);
end
```

Pro Tip

Use `$monitor` for debugging signal changes and `$display` for specific checkpoints.

Q13. Explain the purpose of `timescale` in Verilog.

Answer: `timescale defines the time unit and precision for simulation. Format:

`timescale <time_unit>/<time_precision>

- Time unit: base unit of delay - Time precision: accuracy of simulation time

Code

```
`timescale 1ns/1ps

initial begin
  #5 $display("5ns delay");
  #0.1 $display("0.1ns = 100ps delay");
end
```

Pro Tip

Always include `timescale at the top of simulation files for consistency.

//

//

Q14. What is the difference between `case`

Answer: - `case`: Performs exact bit-by-bit matching. - `casex`: Treats `x` and `z` in case expression as don't care. - `casez`: Treats only `z` as don't care (not `x`).

Code

```
case(sel)
  2'b00: y = a;
  2'b01: y = b;
  default: y = 0;
endcase
```

Pro Tip

Avoid `casex` in synthesizable code since it can hide unknown states during simulation.

Q15. What are parameters in Verilog?

Answer: Parameters are constants defined within modules that can be overridden during instantiation. They are used for making modules reusable and configurable.

Code

```
module counter #(parameter WIDTH = 8) (
  input clk, rst,
  output reg [WIDTH-1:0] q
);
  always @(posedge clk or posedge rst)
    if(rst) q <= 0;
    else q <= q + 1;
endmodule
```

Pro Tip

Use parameters instead of hardcoding values to make designs flexible.

Q16. Explain the difference between `generate` and `for` loops in Verilog.

Answer: - `for` loop inside an `always` block executes during simulation (behavioral). - `generate-for` loop replicates hardware structures during elaboration (structural).

Code

```
genvar i;
generate
  for(i=0; i<4; i=i+1) begin : gen_block
    and g1(y[i], a[i], b[i]);
  end
endgenerate
```

Pro Tip

Use `generate` for hardware replication and `for` for simulation logic.

Q17. What are continuous assignments in Verilog?

Answer: Continuous assignments use `assign` to model combinational logic. They are evaluated whenever the right-hand side operand changes.

Code

```
assign sum = a ^ b;
assign carry = a & b;
```

Pro Tip

Use continuous assignments for simple combinational logic instead of procedural blocks.

Q18. What is the difference between `posedge` and `negedge`?

Answer: - `posedge`: Triggers on rising edge of a signal. - `negedge`: Triggers on falling edge of a signal.
These are primarily used for clocked sequential circuits.

Code

```
always @(posedge clk) q <= d;  // Rising edge
always @(negedge clk) q <= d;  // Falling edge
```

Pro Tip

Most flip-flops are triggered on `posedge`, but some designs use `negedge` for specific timing requirements.

Q19. What is a sensitivity list in Verilog?

Answer: The sensitivity list determines which signals trigger an `always` block. - In combinational logic: all input signals should be included (`always @(*)`). - In sequential logic: typically includes a clock or reset signal.

Code

```
always @(a or b or c)
  y = a & b & c;

always @(posedge clk or posedge rst)
  if(rst) q <= 0; else q <= d;
```

Pro Tip

Use `@(*)` for combinational logic to avoid missing signals.

Q20. Explain blocking vs non-blocking in the context of race conditions.

Answer: Race conditions occur when multiple assignments to the same variable happen in the same simulation cycle. - Blocking (=) executes immediately and can cause unexpected results. - Non-blocking (<=) schedules updates, reducing race hazards in sequential logic.

Code

```
always @(posedge clk) begin
  a = b;     // Blocking
  b = a;     // Race hazard
end
```

Pro Tip

Always use non-blocking (<=) in clocked sequential blocks to avoid races.

Q21. What is the difference between `task` and `function`?

Answer: - **Function**: Returns a single value, executes in zero simulation time. Cannot contain delays. - **Task**: Can return multiple values via outputs, may include delays, event controls, and timing constructs.

Code

```
function [7:0] add;
  input [7:0] a, b;
  add = a + b;
endfunction

task display;
  input [7:0] d;
  $display("Value=%d", d);
endtask
```

Pro Tip

Use functions for combinational logic and tasks for simulation/testbench activities.

Q22. What is event control in Verilog?

Answer: Event control waits for a specific condition or signal change before executing code. Types include: - `@(posedge clk)` → wait for rising edge - `@(negedge clk)` → wait for falling edge - `@(signal)` → wait for signal change

Code

```
always @(posedge clk) q <= d;

initial begin
  @(posedge reset);
  $display("Reset occurred");
end
```

Pro Tip

Event controls are widely used in testbenches to synchronize with signal changes.

Q23. Explain the difference between RTL and Behavioral modeling.

Answer: - **Behavioral modeling**: Describes what the circuit should do using high-level constructs (if, case, loops). May not be directly synthesizable. - **RTL (Register Transfer Level)**: Describes hardware at flip-flop and combinational logic level. Synthesizable into real hardware.

Code

```
// Behavioral
always @(posedge clk)
  if (enable) q <= d;

// RTL
assign q_next = enable ? d : q;
```

Pro Tip

Write RTL for synthesizable hardware, use behavioral for testbenches.

Q24. What is a race condition in Verilog simulation?

Answer: A race condition occurs when the simulation result depends on the execution order of concurrent statements. This usually happens when multiple procedural blocks update the same variable without proper synchronization.

Code

```
always @(posedge clk) a = b;
always @(posedge clk) b = a;
```

Pro Tip

Use non-blocking assignments and proper synchronization to avoid race conditions.

Q25. What is a blocking delay (#) in Verilog?

Answer: A blocking delay (#) introduces a time delay in execution. It suspends the process for a specific simulation time. Delays are not synthesizable and should only be used in testbenches.

Code

```
initial begin
  clk = 0;
  forever #5 clk = ~clk; // Toggle every 5 time units
end
```

Pro Tip

Never use delays in synthesizable RTL code—only use them in testbenches.

Q26. What is the difference between `initial` and `always` blocks?

Answer: - `initial`: Executes only once at the beginning of simulation. Used in testbenches. - `always`: Executes repeatedly whenever the sensitivity condition is triggered. Used for modeling hardware.

Code

```
initial begin
  clk = 0;     // Runs once
end

always #5 clk = ~clk;   // Runs forever
```

Pro Tip

Use `always` for synthesizable hardware; `initial` should be reserved for testbenches.

Q27. What are system tasks in Verilog?

Answer: System tasks are predefined functions used for simulation and debugging. Examples: - `$display` – prints messages - `$monitor` – tracks signals - `$time` – returns current simulation time - `$finish` – ends simulation

Code

```
initial begin
  $display("Simulation start at %0t", $time);
  #100 $finish;
end
```

Pro Tip

System tasks are not synthesizable; they are used only for verification.

Q28. What are the types of delays in Verilog?

Answer: 1. **Inertial delay**: Default delay in gate-level modeling, ignores short glitches. 2. **Transport delay**: Models all signal changes including glitches.

Code

```
assign #5 y = a & b;    // Inertial delay
// Transport delay is modeled in VHDL, but not directly in
   Verilog.
```

Pro Tip

Delays should be avoided in synthesizable RTL—keep them for testbenches.

Q29. What is the difference between `posedge` and `negedge` for resets?

Answer: - `posedge reset`: Active-high reset (triggered when reset goes 0→1). - `negedge reset`: Active-low reset (triggered when reset goes 1→0). Both are used for asynchronous reset in sequential logic.

Code

```
always @(posedge clk or negedge reset_n)
  if(!reset_n) q <= 0;
  else q <= d;
```

Pro Tip

Active-low resets (`negedge`) are common in ASIC design for robustness.

Q30. What is the difference between `defparam` and parameter override?

Answer: - `defparam`: Overrides parameter values from outside the module. - Parameter override during instantiation: Preferred, cleaner, and recommended.

Code

```
module counter #(parameter WIDTH=8) (...);

// Preferred method
counter #(.WIDTH(16)) u1 (...);

// Old method (avoid)
defparam u1.WIDTH = 16;
```

Pro Tip

Always use parameter override during instantiation instead of `defparam`.

Q31. What is the difference between `blocking delay` and `non-blocking delay`?

Answer: - Blocking delay (#): Suspends execution for a specific time. - Non-blocking delay (## in SystemVerilog): Used for cycle-based delays in verification.

Code

```
initial begin
  #5 a = 1;    // Blocking delay
end
```

Pro Tip

In pure Verilog, only blocking delays are available. Use carefully in testbenches.

Q32. What is the difference between packed and unpacked arrays in Verilog?

Answer: - Packed arrays: Treated as vectors, stored contiguously in memory. - Unpacked arrays: Stored separately, like arrays of elements. Packed arrays are synthesizable; unpacked arrays are often used in testbenches.

Code

```
reg [7:0] data_packed [0:3];    // Unpacked array of 8-bit
   vectors
reg [3:0][7:0] data_packed2;    // Packed array
```

Pro Tip

Use packed arrays for RTL and unpacked arrays in verification/testbenches.

Q33. What is the difference between `blocking assignment (=)` and delay control (#)?

Answer: - Blocking assignment (=) executes immediately in sequence. - Delay control (#) introduces time delay before execution. They can be combined in testbenches but are not synthesizable.

Code

```
initial begin
  #5 a = b;    // Assign after 5 time units
end
```

Pro Tip

Avoid delays in RTL. They are useful only in simulation.

Q34. What is the difference between `fork-join` and `begin-end`?

Answer: - `begin-end`: Executes statements sequentially. - `fork-join`: Executes statements in parallel. Used only in simulation/testbenches.

Code

```
initial begin
  a = 1;
  b = 2;
end

initial fork
  a = 1;
  b = 2;
join
```

Pro Tip

Use `fork-join` in testbenches when you want parallel stimulus.

Q35. What are compiler directives in Verilog?

Answer: Compiler directives are special instructions that control compilation. Examples: - `timescale – defines time units - `define – defines macros - `include – includes files

Code

```
`define WIDTH 8
module counter(input clk, output reg [`WIDTH-1:0] q);
...
endmodule
```

Pro Tip

Use `define carefully; prefer parameters for module-specific constants.

Q36. What is the difference between `real`,`integer`,`time` data types?

Answer: - `integer`: 32-bit signed integer. - `real`: Double-precision floating-point number (for simulation). - `time`: 64-bit unsigned integer for simulation time.

Code

```
integer i;
real r;
time t;

initial begin
  i = 10;
  r = 3.14;
  t = $time;
end
```

Pro Tip

Use `integer` for counters, `time` for measuring delays, and `real` only in testbenches.

Q37. What are user-defined primitives (UDP) in Verilog?

Answer: UDPs allow designers to define custom logic functions at the gate level. Two types: - Combinational UDP - Sequential UDP

Code

```
primitive my_and (out, a, b);
  output out;
  input a, b;
  table
    0 0 : 0;
    0 1 : 0;
    1 0 : 0;
    1 1 : 1;
  endtable
endprimitive
```

Pro Tip

UDPs are rarely used in modern RTL but are useful for library modeling.

Q38. What is the difference between `posedge clk` and `@(posedge clk)`?

Answer: - `posedge clk`: Used in sensitivity list of an always block. - `@(posedge clk)`: Explicit event control inside procedural code.

Code

```
always @(posedge clk) q <= d;

initial begin
  @(posedge clk);  // Wait for rising edge
  $display("Clock edge detected");
end
```

Pro Tip

Use `@(posedge clk)` in testbenches, `always @(posedge clk)` in RTL.

Q39. What is the difference between blocking assignment and continuous assignment?

Answer: - Blocking assignment (`=`) is procedural, used inside always/initial blocks. - Continuous assignment (`assign`) is concurrent, used outside procedural blocks.

Code

```
assign y = a & b; // Continuous
always @(a or b)
  y = a & b;      // Blocking
```

Pro Tip

Use `assign` for simple combinational logic; blocking assignment for procedural modeling.

Q40. What is the difference between implicit and explicit nets?

Answer: - Implicit nets: Created automatically if undeclared signals are used. - Explicit nets: Declared explicitly using `wire`.

Code

```
assign y = a & b; // y,a,b implicit nets
wire a, b, y;     // Explicit nets
```

Pro Tip

Always declare nets explicitly to avoid hidden bugs.

Q41. What is synthesis in Verilog?

Answer: Synthesis is the process of converting Verilog RTL code into gate-level netlist for hardware implementation. It excludes non-synthesizable constructs like `initial`, `$display`, delays, etc.

Code

```
always @(posedge clk) q <= d; // Synthesizable
initial $display("Hello");    // Not synthesizable
```

Pro Tip

Always write RTL with synthesizability in mind if targeting ASIC/FPGA.

Q42. What are the main differences between Verilog and SystemVerilog?

Answer: - Verilog: Basic HDL, limited OOP support, less strict checks. - SystemVerilog: Superset of Verilog with OOP, assertions, constrained random, better testbench features.

Code

```
// Verilog
reg a;
// SystemVerilog
logic a;
```

Pro Tip

For RTL, Verilog is enough; for verification, SystemVerilog is preferred.

Q43. What is an FSM in Verilog?

Answer: FSM (Finite State Machine) is a design model with states and transitions. Two types: - Mealy (output depends on state + input) - Moore (output depends only on state)

Code

```
always @(posedge clk) begin
  case(state)
    IDLE: if(start) state <= RUN;
    RUN : if(done) state <= IDLE;
  endcase
end
```

Pro Tip

Use enumerated states for readability and debugging.

Q44. What is the difference between RTL simulation and Gate-level simulation?

Answer: - RTL simulation: Runs on behavioral RTL code. Fast but idealized. - Gate-level simulation: Runs on synthesized netlist with delays. Slower but more accurate.

Code

```
`timescale 1ns/1ps
// RTL sim runs fast, GLS verifies timing with SDF
```

Pro Tip

Run GLS for final sign-off before tape-out.

Q45. What are blocking and non-blocking assignments in FSM design?

Answer: - Use non-blocking (<=) for state transitions and registers. - Use blocking (=) for combinational next-state logic.

Code

```
always @(posedge clk)
  state <= next_state;

always @(*) begin
  case(state)
    S0: next_state = S1;
  endcase
end
```

Pro Tip

Follow the guideline: Non-blocking for sequential, blocking for combinational.

Q46. What is a race around condition in Verilog?

Answer: Occurs when the output depends on the order of execution of concurrent blocks. It leads to nondeterministic simulation results.

Code

```
always @(posedge clk) a = b;
always @(posedge clk) b = a;   // Race condition
```

Pro Tip

Avoid by using non-blocking assignments and proper synchronization.

Q47. What are tristate buffers in Verilog?

Answer: Tristate buffers allow multiple drivers on a bus, with outputs in 0, 1, or high-impedance (Z). Useful in bidirectional bus systems.

Code

```
assign bus = enable ? data : 1'bz;
```

Pro Tip

Avoid tristates in FPGA design; use multiplexers instead.

Q48. What is the difference between RTL and Structural modeling?

Answer: - RTL: Describes design using registers and logic operations. - Structural: Describes design by instantiating gates or modules.

Code

```
// RTL
assign y = a & b;

// Structural
and g1(y, a, b);
```

Pro Tip

RTL is most widely used for synthesis; structural is used in library modeling.

Q49. What is the difference between cycle accurate and event-driven simulation?

Answer: - Event-driven: Simulation occurs on signal changes (Verilog default). - Cycle accurate: Models clock-cycle behavior without intra-cycle details.

Code

```
// Event-driven: updates on signal changes
// Cycle-accurate: useful for system-level modeling
```

Pro Tip

Event-driven is precise; cycle-accurate is faster for large system simulation.

Q50. What are synthesizable and non-synthesizable constructs in Verilog?

Answer: - Synthesizable: `always @(posedge clk)`, `assign`, registers, combinational logic. - Non-synthesizable: `initial`, delays, system tasks, file I/O, fork-join.

Code

```
// Synthesizable
always @(posedge clk) q <= d;

// Non-synthesizable
initial $display("Test");
```

Pro Tip

Keep RTL clean of non-synthesizable constructs to ensure portability.

Q51. Write Verilog code for a positive edge-triggered D Flip-Flop.

Answer: A D Flip-Flop captures input `d` on the rising edge of the clock and stores it in `q`. It may also include asynchronous reset functionality.

Code

```
module d_ff (
  input wire clk, reset, d,
  output reg q
);
  always @(posedge clk or posedge reset) begin
    if (reset)
      q <= 1'b0;
    else
      q <= d;
  end
endmodule
```

Pro Tip

Always use non-blocking assignments (`<=`) in sequential logic.

Q52. Write Verilog code for a JK Flip-Flop.

Answer: A JK Flip-Flop toggles output when both `J` and `K` are high, sets when `J=1, K=0`, and resets when `J=0, K=1`.

Code

```
module jk_ff (
  input wire clk, reset, j, k,
  output reg q
);
  always @(posedge clk or posedge reset) begin
    if (reset)
      q <= 1'b0;
    else begin
      case ({j, k})
        2'b00: q <= q;        // No change
        2'b01: q <= 1'b0;     // Reset
        2'b10: q <= 1'b1;     // Set
        2'b11: q <= ~q;       // Toggle
      endcase
    end
  end
endmodule
```

Pro Tip

JK Flip-Flops are rarely used directly; D-FFs dominate in modern design.

Q53. How to implement a T Flip-Flop in Verilog?

Answer: A T Flip-Flop toggles its state when input `T` is high. Otherwise, it holds its previous state.

Code

```
module t_ff (
  input wire clk, reset, t,
  output reg q
);
  always @(posedge clk or posedge reset) begin
    if (reset)
      q <= 1'b0;
    else if (t)
      q <= ~q;
  end
endmodule
```

Pro Tip

T-FFs are useful in counter design.

Q54. Write Verilog code for a 4-bit Ripple Counter using T Flip-Flops.

Answer: Ripple counters are asynchronous counters where each flip-flop toggles based on the previous one's output.

Code

```
module ripple_counter (
  input wire clk, reset,
  output [3:0] q
);
  t_ff t0 (clk, reset, 1'b1, q[0]);
  t_ff t1 (q[0], reset, 1'b1, q[1]);
  t_ff t2 (q[1], reset, 1'b1, q[2]);
  t_ff t3 (q[2], reset, 1'b1, q[3]);
endmodule
```

Pro Tip

Ripple counters are simple but suffer from propagation delay. Use synchronous counters for high-speed designs.

Q55. Write Verilog code for a 4-bit Synchronous Counter.

Answer: Synchronous counters update all flip-flops simultaneously on the same clock edge.

Code

```
module sync_counter (
  input wire clk, reset,
  output reg [3:0] q
);
  always @(posedge clk or posedge reset) begin
    if (reset)
      q <= 4'b0000;
    else
      q <= q + 1;
  end
endmodule
```

Pro Tip

Synchronous counters are preferred in FPGA/ASIC design due to better timing control.

Q56. Write Verilog code for a Mod-10 Counter.

Answer: A Mod-10 (decade) counter counts from 0 to 9, then resets to 0.

Code

```
module mod10_counter (
  input wire clk, reset,
  output reg [3:0] q
);
  always @(posedge clk or posedge reset) begin
    if (reset)
      q <= 4'b0000;
    else if (q == 4'd9)
      q <= 4'b0000;
    else
      q <= q + 1;
  end
endmodule
```

Pro Tip

Use modulo counters in digital clocks and timers.

Q57. What is the difference between Mealy and Moore FSMs?

Answer: - Moore: Output depends only on the current state. - Mealy: Output depends on both state and input. Mealy FSMs often use fewer states but are more input-sensitive.

Pro Tip

Prefer Moore FSMs for stable outputs; Mealy FSMs are efficient but sensitive to glitches.

Q58. Write Verilog code for a simple Moore FSM (sequence detector for "101").

Answer: This FSM outputs 1 whenever the sequence "101" is detected in the input stream.

Code

```
module seq_detector (
  input wire clk, reset, din,
  output reg dout
);
  typedef enum reg [1:0] {S0, S1, S2, S3} state_t;
  state_t state, next_state;

  always @(posedge clk or posedge reset) begin
    if (reset) state <= S0;
    else state <= next_state;
  end

  always @(*) begin
    case (state)
      S0: next_state = din ? S1 : S0;
      S1: next_state = din ? S1 : S2;
      S2: next_state = din ? S3 : S0;
      S3: next_state = din ? S1 : S0;
      default: next_state = S0;
    endcase
  end

  always @(*) begin
    dout = (state == S3);
  end
endmodule
```

Pro Tip

Sequence detectors are common interview FSM coding problems.

Q59. Write Verilog code for a 4-bit Shift Register.

Answer: A shift register shifts data left/right on each clock edge.

Code

```
module shift_reg (
  input wire clk, reset, d,
  output reg [3:0] q
);
  always @(posedge clk or posedge reset) begin
    if (reset)
      q <= 4'b0000;
    else
      q <= {q[2:0], d}; // Shift left
  end
endmodule
```

Pro Tip

Shift registers are widely used in UARTs and serial-to-parallel conversion.

Q60. Write Verilog code for an Asynchronous Resettable D Flip-Flop.

Answer: This D-FF updates on the rising edge of clock and resets immediately when reset is asserted (regardless of clock).

Code

```
module dff_async_reset (
  input wire clk, reset, d,
  output reg q
);
  always @(posedge clk or posedge reset) begin
    if (reset)
      q <= 1'b0;
    else
      q <= d;
  end
endmodule
```

Pro Tip

Always include reset logic in flip-flops for predictable hardware initialization.

Q61. Write Verilog code for a Synchronous Resettable D Flip-Flop.

Answer: The reset signal only takes effect on the active clock edge.

Code

```
module dff_sync_reset (
  input wire clk, reset, d,
  output reg q
);
  always @(posedge clk) begin
    if (reset)
      q <= 1'b0;
    else
      q <= d;
  end
endmodule
```

Pro Tip

Synchronous resets avoid asynchronous glitches but require the clock to be active.

Q62. Write Verilog code for a 4-bit Up Counter.

Answer: An up counter increments its value on each clock cycle.

Code

```
module up_counter (
  input wire clk, reset,
  output reg [3:0] q
);
  always @(posedge clk or posedge reset) begin
    if (reset)
      q <= 4'b0000;
    else
      q <= q + 1;
  end
endmodule
```

Pro Tip

Use up counters in address generation and event counting.

Q63. Write Verilog code for a 4-bit Down Counter.

Answer: A down counter decrements its value on each clock cycle.

Code

```
module down_counter (
  input wire clk, reset,
  output reg [3:0] q
);
  always @(posedge clk or posedge reset) begin
    if (reset)
      q <= 4'b1111;
    else
      q <= q - 1;
  end
endmodule
```

Pro Tip

Down counters are useful in timers and event countdown circuits.

Q64. Write Verilog code for an Up-Down Counter.

Answer: This counter increments or decrements based on the direction input.

Code

```
module up_down_counter (
  input wire clk, reset, dir,
  output reg [3:0] q
);
  always @(posedge clk or posedge reset) begin
    if (reset)
      q <= 4'b0000;
    else if (dir)
      q <= q + 1;
    else
      q <= q - 1;
  end
endmodule
```

Pro Tip

Add terminal count detection for designing Mod-N up-down counters.

Q65. Write Verilog code for a Ring Counter.

Answer: A ring counter circulates a single '1' across its bits.

Code

```
module ring_counter (
  input wire clk, reset,
  output reg [3:0] q
);
  always @(posedge clk or posedge reset) begin
    if (reset)
      q <= 4'b0001;
    else
      q <= {q[2:0], q[3]};
  end
endmodule
```

Pro Tip

Ring counters are used in sequence generation and timing applications.

Q66. Write Verilog code for a Johnson Counter.

Answer: A Johnson counter (or twisted ring) shifts its inverted last bit into the front.

Code

```
module johnson_counter (
  input wire clk, reset,
  output reg [3:0] q
);
  always @(posedge clk or posedge reset) begin
    if (reset)
      q <= 4'b0000;
    else
      q <= {~q[0], q[3:1]};
  end
endmodule
```

Pro Tip

Johnson counters generate 2N states with N flip-flops.

Q67. Write Verilog code for a Serial-In Serial-Out (SISO) Shift Register.

Answer: SISO registers shift data bit by bit through the chain of flip-flops.

Code

```
module siso_reg (
  input wire clk, reset, d,
  output reg q
);
  reg [3:0] shift;

  always @(posedge clk or posedge reset) begin
    if (reset)
      shift <= 4'b0000;
    else
      shift <= {shift[2:0], d};
  end

  assign q = shift[3];
endmodule
```

Pro Tip

SISO registers are useful in serial communication links.

Q68. Write Verilog code for a Serial-In Parallel-Out (SIPO) Shift Register.

Answer: This register shifts data serially but provides parallel access.

Code

```
module sipo_reg (
  input wire clk, reset, d,
  output reg [3:0] q
);
  always @(posedge clk or posedge reset) begin
    if (reset)
      q <= 4'b0000;
    else
      q <= {q[2:0], d};
  end
endmodule
```

Pro Tip

Used in UART receivers for parallel data recovery.

Q69. Write Verilog code for a Parallel-In Serial-Out (PISO) Shift Register.

Answer: Loads parallel data and then shifts it serially.

Code

```
module piso_reg (
  input wire clk, reset, load,
  input wire [3:0] din,
  output reg dout
);
  reg [3:0] shift;
  always @(posedge clk or posedge reset) begin
    if (reset)
      shift <= 4'b0000;
    else if (load)
      shift <= din;
    else
      shift <= {1'b0, shift[3:1]};
  end
  assign dout = shift[0];
endmodule
```

Pro Tip

PISO registers are useful for data serialization in communication protocols.

Q70. Write Verilog code for a Parallel-In Parallel-Out (PIPO) Register.

Answer: Directly loads parallel inputs and makes them available at outputs.

Code

```
module pipo_reg (
  input wire clk, reset,
  input wire [3:0] din,
  output reg [3:0] q
);
  always @(posedge clk or posedge reset) begin
    if (reset)
      q <= 4'b0000;
    else
      q <= din;
  end
endmodule
```

Pro Tip

PIPO registers are the simplest form of registers used in CPUs.

Q71. Design a Moore FSM that detects three consecutive 1's.

Answer: The FSM outputs high when three 1's appear consecutively in the input stream.

Code

```
module three_ones_detector (
  input wire clk, reset, din,
  output reg dout
);
  typedef enum reg [1:0] {S0, S1, S2, S3} state_t;
  state_t state, next_state;

  always @(posedge clk or posedge reset) begin
    if (reset) state <= S0;
    else state <= next_state;
  end

  always @(*) begin
    case (state)
      S0: next_state = din ? S1 : S0;
      S1: next_state = din ? S2 : S0;
      S2: next_state = din ? S3 : S0;
      S3: next_state = din ? S3 : S0;
    endcase
  end

  always @(*) begin
    dout = (state == S3);
  end
endmodule
```

Pro Tip

FSM-based sequence detectors are standard digital design interview questions.

Q72. Write Verilog code for a Binary to Gray Code Converter.

Answer: Gray codes change only one bit between successive values.

Code

```
module bin2gray (
  input wire [3:0] bin,
  output wire [3:0] gray
);
  assign gray = bin ^ (bin >> 1);
endmodule
```

Pro Tip

Gray codes are widely used in rotary encoders and error reduction.

Q73. Write Verilog code for a Gray to Binary Converter.

Answer: Gray to Binary conversion is recursive: MSB stays same, each next bit is XOR of previous binary bit and current gray bit.

Code

```
module gray2bin (
  input wire [3:0] gray,
  output reg [3:0] bin
);
  integer i;
  always @(*) begin
    bin[3] = gray[3];
    for (i = 2; i >= 0; i = i - 1)
      bin[i] = bin[i+1] ^ gray[i];
  end
endmodule
```

Pro Tip

Gray-Binary conversions are critical in asynchronous FIFO designs.

Q74. Write Verilog code for a 4-bit Comparator.

Answer: A comparator checks if two inputs are equal, greater, or less.

Code

```
module comparator4 (
  input wire [3:0] a, b,
  output wire eq, gt, lt
);
  assign eq = (a == b);
  assign gt = (a > b);
  assign lt = (a < b);
endmodule
```

Pro Tip

Comparators are fundamental building blocks in ALUs.

Q75. Write Verilog code for a Priority Encoder (8-to-3).

Answer: The encoder outputs the binary index of the highest-priority active input.

Code

```
module priority_encoder8 (
  input wire [7:0] din,
  output reg [2:0] dout
);
  always @(*) begin
    casex (din)
      8'b1xxxxxxx: dout = 3'b111;
      8'b01xxxxxx: dout = 3'b110;
      8'b001xxxxx: dout = 3'b101;
      8'b0001xxxx: dout = 3'b100;
      8'b00001xxx: dout = 3'b011;
      8'b000001xx: dout = 3'b010;
      8'b0000001x: dout = 3'b001;
      8'b00000001: dout = 3'b000;
      default: dout = 3'b000;
    endcase
  end
endmodule
```

Pro Tip

Priority encoders are used in interrupt handling circuits.

Q76. What is the difference between latch and flip-flop in Verilog?

Answer: - A latch is level-sensitive (output changes when enable is active). - A flip-flop is edge-triggered (output changes only at clock edge). Flip-flops are widely used in synchronous design, while latches are avoided due to timing complexity.

Example: D Latch vs D Flip-Flop

```
module d_latch(input d, en, output reg q);
  always @(*) begin
    if (en) q = d;
  end
endmodule

module d_ff(input d, clk, output reg q);
  always @(posedge clk) begin
    q <= d;
  end
endmodule
```

Q77. Write Verilog code for a JK Flip-Flop.

Answer: A JK flip-flop toggles its output when both inputs are high, otherwise behaves like SR flip-flop with no invalid state.

JK Flip-Flop

```
module jk_ff(input j, k, clk, output reg q);
  always @(posedge clk) begin
    case ({j,k})
      2'b00: q <= q;      // hold
      2'b01: q <= 0;      // reset
      2'b10: q <= 1;      // set
      2'b11: q <= ~q;     // toggle
    endcase
  end
endmodule
```

Q78. Explain synchronous vs asynchronous counter.

Answer: - **Synchronous counter**: All flip-flops are triggered by the same clock, so outputs change simultaneously. - **Asynchronous counter**: Clock is passed through flip-flop outputs, causing propagation delay.

Example: 4-bit Synchronous Counter

```
module sync_counter(input clk, rst, output reg [3:0] q);
  always @(posedge clk or posedge rst) begin
    if (rst) q <= 0;
    else q <= q + 1;
  end
endmodule
```

Q79. Design a 4-bit Johnson Counter in Verilog.

Answer: A Johnson counter feeds back the inverted output of the last flip-flop into the first, generating 2n unique states.

Johnson Counter

```
module johnson_counter(input clk, rst, output reg [3:0] q);
  always @(posedge clk or posedge rst) begin
    if (rst) q <= 0;
    else q <= {~q[0], q[3:1]};
  end
endmodule
```

Q80. Write Verilog code for a 2:1 multiplexer.

Answer: A 2:1 MUX selects one of two inputs based on the select signal.

2:1 Multiplexer

```
module mux2to1(input a, b, sel, output y);
  assign y = sel ? b : a;
endmodule
```

Q81. Write Verilog code for a 4:1 multiplexer using case statement.

Answer: We can implement a 4:1 MUX using behavioral modeling with a case statement.

4:1 Multiplexer

```
module mux4to1(input [3:0] d, input [1:0] sel, output reg y);
  always @(*) begin
    case(sel)
      2'b00: y = d[0];
      2'b01: y = d[1];
      2'b10: y = d[2];
      2'b11: y = d[3];
    endcase
  end
endmodule
```

Q82. What is the difference between Moore and Mealy FSM?

Answer: - Moore FSM: Output depends only on state. - Mealy FSM: Output depends on both state and input (can react faster).

Moore vs Mealy FSM Diagram

- Moore: output = f(state) - Mealy: output = f(state, input)

Q83. Write Verilog code for a simple Moore FSM (sequence detector 101).

Answer: This FSM outputs 1 whenever the sequence "101" is detected in serial input.

Moore FSM: Sequence Detector

```
module seq101_moore(input clk, rst, in, output reg out);
  reg [1:0] state, next;
  parameter S0=0, S1=1, S2=2, S3=3;

  always @(posedge clk or posedge rst)
    if (rst) state <= S0;
    else state <= next;

  always @(*) begin
    case(state)
      S0: next = in ? S1 : S0;
      S1: next = in ? S1 : S2;
      S2: next = in ? S3 : S0;
      S3: next = in ? S1 : S2;
    endcase
  end

  always @(*) out = (state == S3);
endmodule
```

Q84. Explain synchronous reset vs asynchronous reset.

Answer: - **Synchronous reset**: Reset occurs only at the clock edge (better timing control). - **Asynchronous reset**: Reset is immediate (even without clock), but can cause metastability if not released synchronously.

Example: Synchronous Reset D-FF

```
always @(posedge clk) begin
  if (rst) q <= 0;
  else q <= d;
end
```

Q85. What is the difference between synthesizable and non-synthesizable code?

Answer: - Synthesizable: Code that can be converted into hardware (e.g., ‘always @(*)‘, flip-flops, muxes). - Non-synthesizable: Testbench constructs, delays (‘10‘), system tasks (‘*display*‘).

Q86. Write Verilog code for a 3-to-8 decoder.

Answer: A decoder activates one output line based on binary input.

3-to-8 Decoder

```
module decoder3to8(input [2:0] a, output reg [7:0] y);
  always @(*) begin
    y = 8'b0;
    y[a] = 1;
  end
endmodule
```

Q87. Write Verilog code for a 4-bit comparator.

Answer: A comparator compares two binary numbers and outputs greater, less, or equal.

4-bit Comparator

```
module comp4(input [3:0] a, b,
             output lt, gt, eq);
  assign lt = (a < b);
  assign gt = (a > b);
  assign eq = (a == b);
endmodule
```

Q88. Explain synthesizable for-loop in Verilog.

Answer: A synthesizable loop is statically unrolled by the synthesis tool. Example: generating repetitive hardware like adders.

Example: Generate Block

```
genvar i;
generate
  for (i=0; i<8; i=i+1) begin: and_gates
    and g(y[i], a[i], b[i]);
  end
endgenerate
```

Q89. Write Verilog code for an 8-bit shift register.

Answer: Shift registers are used for serial-to-parallel or parallel-to-serial conversions.

8-bit Shift Register

```
module shift_reg(input clk, rst, din,
                 output reg [7:0] q);
  always @(posedge clk or posedge rst) begin
    if (rst) q <= 0;
    else q <= {q[6:0], din};
  end
endmodule
```

Q90. What is difference between blocking delay and non-blocking delay?

Answer: - Blocking delay (‘10 a=1;‘) halts execution for 10 time units. - Non-blocking delay is not synthesizable and is mainly used in testbenches.

Q91. Write Verilog code for a ring counter.

Answer: A ring counter rotates a single ‘1’ through flip-flops.

4-bit Ring Counter

```
module ring_counter(input clk, rst,
                    output reg [3:0] q);
  always @(posedge clk or posedge rst) begin
    if (rst) q <= 4'b0001;
    else q <= {q[2:0], q[3]};
  end
endmodule
```

Q92. Explain metastability in flip-flops.

Answer: Metastability occurs when setup or hold times are violated, causing unpredictable output. Mitigation: Use synchronizers (two-stage flip-flops) for async signals.

Q93. Write Verilog code for Gray Code counter.

Answer: Gray code ensures only one bit changes between successive states.

Gray Code Counter

```
module gray_counter(input clk, rst,
                    output reg [3:0] q);
  always @(posedge clk or posedge rst) begin
    if (rst) q <= 0;
    else q <= (q >> 1) ^ q;
  end
endmodule
```

Q94. Explain race conditions in Verilog.

Answer: Race condition occurs when multiple events happen at the same simulation time and ordering is undefined. Avoid by using non-blocking assignments in sequential logic.

Q95. Write Verilog code for a Priority Encoder.

Answer: Priority encoder outputs the highest-order active input.

Priority Encoder

```
module priority_enc(input [3:0] d, output reg [1:0] y);
  always @(*) begin
    casex(d)
      4'b1xxx: y = 2'b11;
      4'b01xx: y = 2'b10;
      4'b001x: y = 2'b01;
      4'b0001: y = 2'b00;
      default: y = 2'b00;
    endcase
  end
endmodule
```

Q96. Write Verilog code for SR Flip-Flop.

Answer: SR flip-flop sets and resets output, with invalid state when both inputs are 1.

SR Flip-Flop

```
module sr_ff(input s, r, clk, output reg q);
  always @(posedge clk) begin
    if (s & ~r) q <= 1;
    else if (~s & r) q <= 0;
    else if (s & r) q <= 1'bx; // invalid
  end
endmodule
```

Q97. Write Verilog code for T Flip-Flop.

Answer: T flip-flop toggles its output when input T=1.

T Flip-Flop

```
module t_ff(input t, clk, output reg q);
  always @(posedge clk) begin
    if (t) q <= ~q;
  end
endmodule
```

Q98. Explain difference between *display*and**monitor.**

Answer: - `$display`: Prints once when executed. - `$monitor`: Continuously prints whenever variable changes.

Q99. Write Verilog code for Binary to Gray Converter.

Answer:

Binary to Gray Converter

```
module bin2gray(input [3:0] b, output [3:0] g);
  assign g = (b >> 1) ^ b;
endmodule
```

Q100. Write Verilog code for Gray to Binary Converter.

Answer:

Gray to Binary Converter

```
module gray2bin(input [3:0] g, output reg [3:0] b);
  always @(*) begin
    b[3] = g[3];
    b[2] = g[3] ^ g[2];
    b[1] = b[2] ^ g[1];
    b[0] = b[1] ^ g[0];
  end
endmodule
```

Summary

This document has compiled more than 100 carefully curated Verilog interview questions with comprehensive answers, code examples, and practical insights. It covers a wide spectrum of topics including basic Verilog constructs, modeling styles, flip-flops, counters, FSMs, and hardware design practices.

The intention of this guide is not just to prepare you for interviews, but also to strengthen your fundamentals in Verilog and digital design. If you revise these questions thoroughly, you will gain the confidence to approach real-world interviews with clarity and precision.

Acknowledgment & Thanks

Thank you for taking the time to go through this guide. I truly hope it helps you in cracking interviews and building a strong career in the VLSI industry.

If at any point you face difficulties in understanding concepts, or if you are looking for a mock interview session to simulate real interview scenarios and improve your confidence, feel free to reach out.

I will be more than happy to guide you personally so that you can unlock better career opportunities.

—

Contact Information

For further guidance, counseling, or mock interview sessions, please contact me at:

- **Phone:** +91 9016531681
- **Email:** infoex31@gmail.com

Stay consistent, stay curious, and keep practicing. Your dedication today will shape your future in the semiconductor industry.

— Best wishes for your journey ahead —